

Poglavje 5

Prirejanja in pokritja, pretoki in prerezi

5.5 Izrek o celih rešitvah za problem največjega pretoka

Za Ford-Fulkersonov algoritem obstajajo podatki (z iracionalnimi prepustnostmi), pri katerih teče v neskončnost. Pokažimo, da se to v primeru, ko so prepustnosti cele ali racionalne, ne more zgoditi.

Izrek 1 Če so v omrežju (G, s, t, c) vse prepustnosti $c(i, j)$ cela števila, se algoritem FF konča in vrne celoštevilski največji pretok.

Dokaz: Začetni pretok f v algoritmu FF je celoštevilski. Če je na nekem koraku pretok f celoštevilski, so vse residualne prepustnosti $r(i, j) = c(i, j) - f(i, j)$ cela števila. Zato je tudi število $d = \min_{(i,j) \in E(P)} r(i, j)$ celo in iz $d > 0$ sledi $d \geq 1$. Povečujoči pretok

$$f_P(i, j) = \begin{cases} d, & (i, j) \in E(P) \\ -d, & (j, i) \in E(P) \\ 0, & \text{sicer} \end{cases}$$

je celoštevilski, torej je tak tudi novi pretok $f + f_P$ in velja:

$$|f + f_P| = |f| + |f_P| = |f| + d \geq |f| + 1.$$

Z indukcijo po številki koraka sledi, da je pretok f na vsakem koraku celoštevilski in da se na vsakem koraku poveča vsaj za 1.

Naj bo (A, B) poljuben prerez v (G, s, t, c) . Potem je $|f| \leq c(A, B) \in \mathbb{N}$ za vse pretoke f v (G, s, t, c) . Torej se FF konča po največ $c(A, B)$ korakih in vrne celoštevilski največji pretok. $\square$

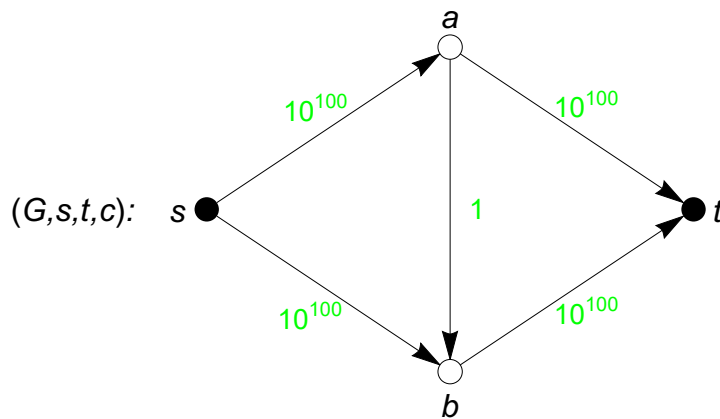
Posledica 1 (izrek o celih rešitvah) Če so v omrežju (G, s, t, c) vse prepustnosti $c(i, j)$ cela števila, v (G, s, t, c) obstaja celoštevilski največji pretok.

Posledica 2 (izrek o racionalnih rešitvah) Če so v omrežju (G, s, t, c) vse prepustnosti $c(i, j)$ racionalna števila, v (G, s, t, c) obstaja racionalen največji pretok.

Dokaz: Naj bo k najmanjši skupni imenovalec vseh prepustnosti $c(i, j)$. Uporabimo FF na omrežju (G, s, t, kc) . Po izreku 1 se FF konča in vrne celoštevilski največji pretok f . Potem je očitno f/k racionalen največji pretok v (G, s, t, c) . $\square$

Če so prepustnosti cela števila, se torej algoritem FF gotovo ustavi, a v najslabšem primeru se pretok na vsakem koraku povečuje le za 1. Oglejmo si primer zelo majhnega omrežja, na katerem lahko zaradi velikih prepustnosti FF teče zelo dolgo.

Zgled 1 Poiščimo največji pretok v omrežju na sliki.



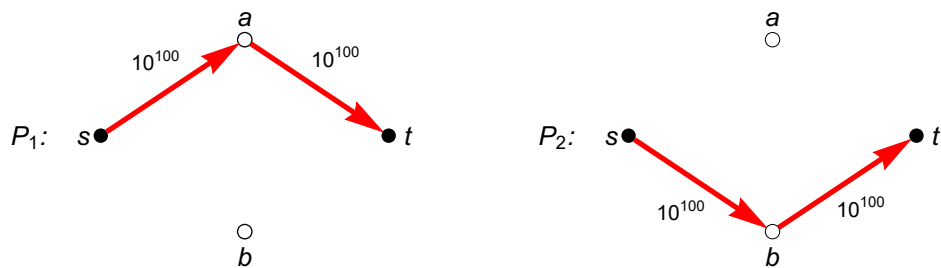
Začnimo s pretokom $f = 0$. Algoritem FF lahko poteka na različne načine, saj v residualnem omrežju v splošnem obstaja več usmerjenih poti $s \rightarrow t$.

MOŽNOST A:

1. korak: FF izbere povečujočo pot $P_1 = sat$ z najmanjšo prepustnostjo $d = 10^{100}$. Dobimo pretok f_1 velikosti 10^{100} , povezavi (s, a) in (a, t) sta zasičeni.

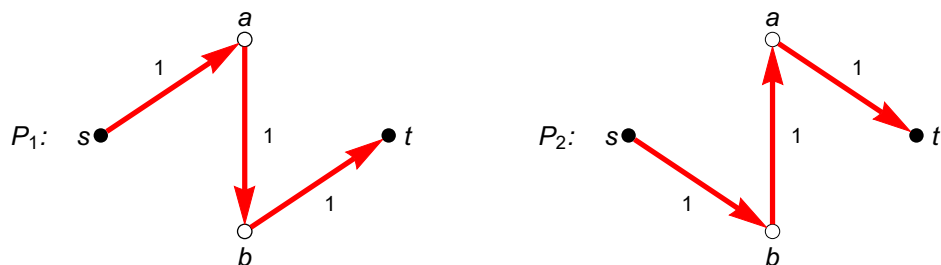
2. korak: Edina povečujoča pot je $P_2 = sbt$, spet z najmanjšo prepustnostjo $d = 10^{100}$. Dobimo pretok f_2 velikosti $2 \cdot 10^{100}$, povezave (s, a) , (a, t) , (s, b) , (b, t) so zasičene. V residualnem omrežju ni več usmerjenih poti $s \rightarrow t$ in FF se konča v dveh korakih.

5.5. IZREK O CELIH REŠITVAH ZA PROBLEM NAJVEČJEGA PRETOKA3

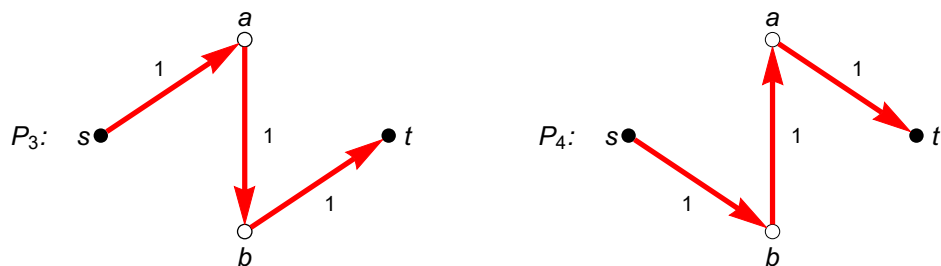


MOŽNOST B:

1. korak: FF izbere povečujočo pot $P_1 = s \rightarrow a \rightarrow t$ z najmanjšo prepustnostjo $d = 1$. Dobimo pretok f_1 velikosti 1, povezava (a, b) je zasičena.
2. korak: FF izbere povečujočo pot $P_2 = s \rightarrow b \rightarrow t$ z najmanjšo prepustnostjo $d = 1$. Dobimo pretok f_2 velikosti 2.



3. korak: FF izbere povečujočo pot $P_3 = s \rightarrow a \rightarrow b \rightarrow t$ z najmanjšo prepustnostjo $d = 1$. Dobimo pretok f_3 velikosti 3, povezava (a, b) je zasičena.
4. korak: FF izbere povečujočo pot $P_4 = s \rightarrow b \rightarrow a \rightarrow t$ z najmanjšo prepustnostjo $d = 1$. Dobimo pretok f_4 velikosti 4.



Tako nadaljujemo, dokler se ne zasitijo povezave (s, a) , (a, t) , (s, b) in (b, t) . Takrat v residualnem omrežju ni več usmerjenih poti $s \rightarrow t$ in FF se konča z enakim pretokom velikosti $2 \cdot 10^{100}$ kot pri možnosti A, a šele po $2 \cdot 10^{100}$ korakih!

V resnici je algoritem FF le shema iskanja največjega pretoka, saj ne določa, katere povečujoče poti naj izberemo, kadar jih je na razpolago več. Algoritmi, ki

se uporabljajo v praksi in so zelo učinkoviti (npr. Edmonds–Karpov, Diničev ali Tarjanov), na vsakem koraku uporabljajo le ***najkrajše*** povečujoče poti (kot smo storili v gornjem zgledu pri možnosti A). Kot se izkaže, potem število korakov ni odvisno od prepustnosti povezav, ampak le od števila vozlišč in števila povezav grafa G .

Poglavje 6

Najkrajše poti

6.1 Najkrajše poti iz skupnega izhodišča

V danem usmerjenem grafu G brez uteži želimo poiskati vse najkrajše poti od danega izhodišča $s \in V(G)$ do vseh drugih vozlišč grafa G .

Za reševanje uporabimo *pregled grafa G v širino* oziroma *pregled po slojih*, kjer za $k = 0, 1, 2, \dots$ sloj k sestavljajo vozlišča, dosegljiva iz izhodišča s v k korakih. Množico vozlišč, dosegljivih iz vozlišča u v enem koraku, imenujmo $\text{otroci}(u)$. Sloj 0 vsebuje le vozlišče s , sloj 1 vozlišča iz množice $\text{otroci}(s)$, sloj 2 vozlišča, dosegljiva iz s v dveh korakih itd. Že pregledana vozlišča hranimo v *vrsti*, t.j. seznamu, v katerega elemente vstavljamo na koncu, odstranjujemo pa jih na začetku (po načelu FIFO oziroma *prvi noter, prvi ven*). Pri tem uporabljamo naslednje ukaze:

```
ustvari(vrsta)    ... ustvari prazno vrsto
vstavi(vrsta, u)  ... vstavi  $u$  na konec vrste
odstrani(vrsta)   ... odstrani prvi element vrste in ga vrne kot rezultat
prazna(vrsta)     ... vrne true, če je vrsta prazna, in false, sicer
```

Definicija 1 *Usmerjen graf je acikličen, če ne vsebuje usmerjenih ciklov.*

Iskanje najkrajših poti s pregledom v širino

Podatki: usmerjen graf G , vozlišče $s \in V(G)$

Rezultat: usmerjeni aciklični graf L_G vseh najkrajših poti od s do vseh drugih vozlišč grafa G

Postopek:

```
ustvari(vrsta)
sloj(s) = 0
vstavi(vrsta, s)
```

```

 $V(L_G) = \{s\}$ 
 $E(L_G) = \emptyset$ 
dokler vrsta ni prazna ponavljaj
     $u = \text{odstrani}(\text{vrsta})$ 
    za vse  $v \in \text{otroci}(u)$  ponovi
        če  $\text{sloj}(v)$  ni definiran potem
             $\text{sloj}(v) = \text{sloj}(u) + 1$ 
             $V(L_G) = V(L_G) \cup \{v\}$ 
             $E(L_G) = E(L_G) \cup \{uv\}$ 
             $\text{vstavi}(\text{vrsta}, v)$ 
        sicer
            če  $\text{sloj}(v) > \text{sloj}(u)$  potem  $E(L_G) = E(L_G) \cup \{uv\}$ 
vrni  $L_G$ 

```

Časovna zahtevnost: $\mathcal{O}(m)$, kjer je $m = |E(G)|$

6.2 Najcenejše poti iz skupnega izhodišča

V danem usmerjenem grafu G z **utežmi na povezavah** želimo poiskati vse najcenejše poti od danega izhodišča $s \in V(G)$ do vseh drugih vozlišč grafa G .

Oznake: Naj bo G usmerjen graf z utežmi $c_{ij} \geq 0$ za vse povezave $ij \in E(G)$.

1. Ceno prehoda S v grafu G označimo z $|S|$:

$$|S| = \sum_{ij \in E(S)} c_{ij}.$$

2. Naj bo P pot v grafu G . Za $u, v \in V(P)$, kjer je vozlišče u predhodnik vozlišča v na poti P , s $P_{u \rightarrow v}$ označimo odsek poti P od u do v .

Trditev 1 Naj bo P najcenejša pot od a do b v grafu G . Če je vozlišče u predhodnik vozlišča v na poti P , je $P_{u \rightarrow v}$ najcenejša pot od u do v v grafu G .

Dokaz: Predpostavimo, da je R pot od u do v v grafu G , ki je cenejša od poti $P_{u \rightarrow v}$. Potem je $S = P_{a \rightarrow u} R P_{v \rightarrow b}$ sprehod od a do b in velja

$$|S| = |P_{a \rightarrow u}| + |R| + |P_{v \rightarrow b}| < |P_{a \rightarrow u}| + |P_{u \rightarrow v}| + |P_{v \rightarrow b}| = |P|.$$

Če na sprehodu S izpustimo vse cikle, dobimo pot Q od a do b , za katero je $|Q| \leq |S| < |P|$, kar pa ni mogoče, saj je P najcenejša pot od a do b . Torej takšna pot R ne obstaja in je $P_{u \rightarrow v}$ res najcenejša pot od u do v v grafu G . $\square$

Dijkstrov algoritem

Podatki: usmerjen graf $G = (V, E)$, vozlišče $s \in V$, cene $c_{ij} \geq 0$ za vse $ij \in E$.

Rezultat: 1. tabela $d[i]$ cen najcenejših poti $s \rightarrow i$ za vse $i \in V$ (če take poti ni, je $d[i] = \infty$)

2. drevo najcenejših poti $s \rightarrow i$ za vse $i \in V$, podano s tabelo kazalcev oče $[i]$ (če take poti ni ali če je $i = s$, je oče $[i] = \text{nil}$)

Postopek:

1. $X = \emptyset$ // množica vozlišč, do katerih že poznamo najcenejšo pot iz s //
2. $d[s] = 0$, oče $[s] = \text{nil}$
3. za vse $j \in V \setminus \{s\}$ ponovi
4. $d[j] = \infty$, oče $[j] = \text{nil}$
5. dokler $X \neq V$ ponavljaaj
6. naj bo $i \in V \setminus X$ vozlišče z min. vrednostjo $d[i]$
7. $X = X \cup \{i\}$
8. za vse $j \in V \setminus X$, za katere je $ij \in E$, ponovi
9. če $d[i] + c_{ij} < d[j]$ potem
10. oče $[j] = i$
11. $d[j] = d[i] + c_{ij}$

Trditev 2 Za vsako vozlišče $j \in V$ se vrednost $d[j]$ na vsakem koraku Dijkstrovega algoritma kvečjemu zmanjša.

Dokaz: novi $d[j] = \min\{d[i] + c_{ij}, \text{stari } d[j]\} \leq \text{stari } d[j] \quad \checkmark$

□

Trditev 3 Dijkstrov algoritem je pravilen.

Dokaz: Množica X je na začetku prazna, na vsaki ponovitvi zanke 5 – 11 pa ji v vrstici 7 dodamo novo vozlišče iz V . Po $|V|$ korakov je torej $X = V$, kar je pogoj za izstop iz te zanke. Zaključimo, da se Dijkstrov algoritem vselej ustavi.

Nekoliko več dela je z dokazom pravilnosti rezultata, ki ga algoritem vrne. Trdimo, da za vse $k \in \mathbb{N}$ na k -ti ponovitvi vrstice 5 velja naslednje:

- (a) za vse $j \in V \setminus \{s\}$ z $d[j] < \infty$:
 - (a1) oče $[j] \in X$,
 - (a2) zaporedje vozlišč $z = (j, \text{oče}[j], \text{oče}[\text{oče}[j]], \dots)$ se konča z vozliščem s ,
 - (a3) cena poti $s \rightarrow j$, ki jo določa zaporedje z , je $d[j]$, in
- (b) za vse $i \in X$, za katere obstaja pot $s \rightarrow i$, je $d[i]$ cena najcenejše poti $s \rightarrow i$.

Trditev dokažimo z indukcijo po k .

$k = 1$: Na 1. ponovitvi vrstice 5 je $d[j] = \infty$ za vse $j \in V \setminus \{s\}$ in $X = \emptyset$, torej trditvi (a) in (b) veljata na prazno.

$k \rightarrow k + 1$: Predpostavimo, da sta na k -ti ponovitvi vrstice 5 trditvi (a) in (b) veljali, in pokažimo, da potem veljata tudi na $(k + 1)$ -vi ponovitvi. Trditev (a) je treba dokazati le za tiste $j \in V \setminus X$, za katere je $ij \in E$ in $d[i] + c_{ij} < d[j]$ (za druge velja po indukcijski predpostavki).

(a1): V vrstici 7 damo i v X , v vrstici 10 pa postavimo $\text{oče}[j] = i$. Torej $\text{oče}[j] \in X$. $\checkmark$

(a2): Ker je $\text{oče}[j] = i$, je po indukcijski predpostavki $(j, \text{oče}[j], \text{oče}[\text{oče}[j]], \dots) = (j, i, \text{oče}[i], \dots, s)$, torej (a2) še vedno velja.

(a3): Cena poti $(j, i, \text{oče}[i], \dots, s)$ je enaka $c_{ij} + \text{cena poti } (i, \text{oče}[i], \dots, s)$, to pa je po indukcijski predpostavki in vrstici 11 enako $c_{ij} + d[i] = d[j]$.

(b): Trditev zadošča dokazati za i iz vrstice 6 (za druge elemente množice X velja po indukcijski predpostavki). Naj bo P najcenejša pot $s \rightarrow i$. Naj bo j prvo vozlišče na P zunaj starega X , in naj bo u neposredni predhodnik j na P . Ko smo u (kot tedanji i) vstavili v X , je veljalo

$$d[j] \leq d[u] + c_{uj}. \quad (6.1)$$

Kasneje se je po trditvi 2 vrednost $d[j]$ kvečjemu zmanjšala, torej (6.1) še vedno velja. Zato sledi

$$\begin{aligned} |P| &= |P_{s \rightarrow u}| + c_{uj} + |P_{j \rightarrow i}| = d[u] + c_{uj} + |P_{j \rightarrow i}| \\ &\geq d[u] + c_{uj} \geq d[j] \geq d[i], \end{aligned} \quad (6.2)$$

kjer smo pri drugem enačaju uporabili trditev 1 in točko (b) indukcijske predpostavke, pri prvem neenačaju nenegativnost cen c , pri drugem neenačaju neenačbo (6.1) in pri tretjem izbiri vozlišča i v vrstici 6. Po (a3) je $d[i]$ cena neke poti $s \rightarrow i$, medtem ko je P najcenejša pot $s \rightarrow i$. Torej je $d[i] \geq |P|$, od tod pa po (6.2) sledi $d[i] = |P|$, kar pomeni, da je $d[i]$ res cena najcenejše poti $s \rightarrow i$ v G .

Ko se zanka 5 – 11 izteče, je $X = V$, zato po točki (b) za vse $i \in V$ velja

$$d[i] = \begin{cases} \text{cena najcenejše poti } s \rightarrow i, & \text{če taka pot obstaja,} \\ \infty, & \text{sicer.} \end{cases}$$

Po točki (a3) je v primeru, ko obstaja pot $s \rightarrow i$, najkrajša taka pot določena z zaporedjem vozlišč $(i, \text{oče}[i], \text{oče}[\text{oče}[i]], \dots, s)$. $\square$

Ocenimo še časovno zahtevnost Dijkstrovega algoritma. Naj bo $n = |V|$ število vozlišč grafa G . Zanka 5 – 11 se izvrši n -krat. Za iskanje vozlišča i z najmanjšo ceno v vrstici 6 potrebujemo največ $n - 1$ korakov, zanka 8 – 11 pa se izvrši največ $(n - 1)$ -krat. Skupaj potrebujemo torej največ $\mathcal{O}(n(2(n - 1))) = \mathcal{O}(n^2)$ korakov.

6.3 Najcenejše poti med vsemi pari vozlišč

V danem usmerjenem grafu G z **utežmi na povezavah** želimo poiskati najcenejše poti od i do j za vse pare vozlišč $(i, j) \in V(G) \times V(G)$. Ogledali si bomo Floyd-Warshallov algoritem, ki uporablja dinamično programiranje in ima časovno zahtevnost $\mathcal{O}(n^3)$, kjer je $n = |V(G)|$.

Označimo vozlišča grafa G z $1, 2, \dots, n$. Za $i, j, k \in \{1, 2, \dots, n\}$ naj bo $d_{ij}^{(k)}$ cena najcenejše poti $i \rightarrow j$ z *notranjimi vozlišči iz množice* $\{1, 2, \dots, k\}$ (pri čemer so notranja vozlišča poti vsa njena vozlišča razen začetnega in končnega). Naj bo P takšna pot; potem obstajata dve možnosti:

a) k ni notranje vozlišče poti P : V tem primeru je $|P| = d_{ij}^{(k-1)}$.

b) k je notranje vozlišče poti P : V tem primeru je $|P| = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$, saj k ni notranje vozlišče odsekov $P_{i \rightarrow k}$ in $P_{k \rightarrow j}$, ki sta najcenejši poti $i \rightarrow k$ in $k \rightarrow j$ z notranjimi vozlišči iz množice $\{1, 2, \dots, k-1\}$.

Ker je P najcenejša pot $i \rightarrow j$ z notranjimi vozlišči iz množice $\{1, 2, \dots, k\}$, v obeh primerih velja

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}. \quad (6.3)$$

Očitno je $d_{ij}^{(0)} = c(i, j)$ za vse $i, j \in \{1, 2, \dots, n\}$. Od tod pa lahko z rekurzivno formulo (6.3) po vrsti izračunamo števila $d_{ij}^{(1)}, d_{ij}^{(2)}, \dots, d_{ij}^{(n)}$. Potem so cene najcenejših poti $i \rightarrow j$ v grafu G , ki jih iščemo, enake $d_{ij}^{(n)}$ za vse $i, j \in \{1, 2, \dots, n\}$.

Floyd-Warshallov algoritem

Podatki: usmerjen graf $G = (V, E)$, kjer je $V = \{1, 2, \dots, n\}$

cene povezav $c: V \times V \rightarrow \mathbb{R} \cup \{\infty\}$, kjer za vse $i, j \in V$, $i \neq j$, velja:

$$c(i, i) = 0 \text{ in } c(i, j) = \infty \iff ij \notin E$$

predpostavka: graf G ne vsebuje negativnih ciklov

Rezultat: tabela cen $d_{ij}^{(n)}$ najcenejših poti $i \rightarrow j$ za vse $(i, j) \in V \times V$

Postopek:

za $i = 1$ do n ponovi

za $j = 1$ do n ponovi

$$d_{ij}^{(0)} = c(i, j)$$

za $k = 1$ do n ponovi

za $i = 1$ do n ponovi

za $j = 1$ do n ponovi

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

Pri računanju elementov $d_{ij}^{(k)}$ si lahko nekaj dela prihranimo.

1. Ker v G ni negativnih ciklov, so diagonalni elementi matrik $D^{(k)}$ ves čas enaki 0: $d_{ii}^{(k)} = 0$ za vse $i, k \in \{1, 2, \dots, n\}$.
2. Na k -tem koraku se elementi k -te vrstice in k -tega stolpca ne spremenijo:

$$\begin{aligned} d_{kj}^{(k)} &= \min\{d_{kj}^{(k-1)}, d_{kk}^{(k-1)} + d_{kj}^{(k-1)}\} = \min\{d_{kj}^{(k-1)}, d_{kj}^{(k-1)}\} = d_{kj}^{(k-1)} \\ d_{ik}^{(k)} &= \min\{d_{ik}^{(k-1)}, d_{ik}^{(k-1)} + d_{kk}^{(k-1)}\} = \min\{d_{ik}^{(k-1)}, d_{ik}^{(k-1)}\} = d_{ik}^{(k-1)} \end{aligned}$$

3. Na k -tem koraku se vrstice in stolpci, ki imajo na k -tem mestu element ∞ , ne spremenijo:

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} = \min\{d_{ij}^{(k-1)}, \infty\} = d_{ij}^{(k-1)}$$

Gornji algoritem nam vrne cene najcenejših poti med vsemi pari vozlišč grafa G , ne pa tudi poti samih. Dopolnimo ga tako, da si na vsakem koraku zapomnimo kazalec na očeta vozlišča j na trenutno najcenejši poti $i \rightarrow j$.

Dopolnjeni Floyd-Warshallov algoritem

Podatki: usmerjen graf $G = (V, E)$, kjer je $V = \{1, 2, \dots, n\}$
cene povezav $c: V \times V \rightarrow \mathbb{R} \cup \{\infty\}$, kjer za vse $i, j \in V$, $i \neq j$, velja:

$$c(i, i) = 0 \text{ in } c(i, j) = \infty \iff ij \notin E$$

predpostavka: graf G ne vsebuje negativnih ciklov

Rezultat: 1. tabela cen $d_{ij}^{(n)}$ najcenejših poti $i \rightarrow j$ za vse $(i, j) \in V \times V$
2. tabela oče $_{ij}^{(n)}$ neposrednih predhodnikov vozlišča j
na najcenejši poti $i \rightarrow j$ za vse $(i, j) \in V \times V$

Postopek:

```

za  $i = 1$  do  $n$  ponovi
  za  $j = 1$  do  $n$  ponovi
     $d_{ij}^{(0)} = c(i, j)$ 
    če  $c(i, j) < \infty$  potem oče $_{ij}^{(0)} = i$  sicer oče $_{ij}^{(0)} = \text{nil}$ 
  za  $k = 1$  do  $n$  ponovi
    za  $i = 1$  do  $n$  ponovi
      za  $j = 1$  do  $n$  ponovi
        če  $d_{ik}^{(k-1)} + d_{kj}^{(k-1)} < d_{ij}^{(k-1)}$  potem

```

$$\begin{aligned}
 d_{ij}^{(k)} &= d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\
 \text{oče}_{ij}^{(k)} &= \text{oče}_{kj}^{(k-1)} \\
 \text{sicer} \\
 d_{ij}^{(k)} &= d_{ij}^{(k-1)} \\
 \text{oče}_{ij}^{(k)} &= \text{oče}_{ij}^{(k-1)}
 \end{aligned}$$

Ocenimo še časovno zahtevnost Floyd-Warshallovega algoritma. Telo trojne zanke se izvede n^3 -krat, torej je število operacij reda velikosti $\mathcal{O}(n^3)$. Enako časovno zahtevnost dobimo sicer tudi z n -kratno uporabo Dijkstrovega algoritma, kjer začetno vozlišče s preteče vsa vozlišča grafa, a Floyd-Warshallov algoritem deluje tudi v primeru negativnih uteži $c(i, j)$.